Sorting Algorithms

Data Structures

Sorting

- ***Sorting*** is a process that organizes a collection of data into either ascending or descending order.
- Formally
  - Input: A sequence of n numbers <a1,a2,…,an>
  - Output: A reordering <a'1,a'2,…,a'n> of the sequence such that a'1 ≤ a'2 ≤ … ≤ a'n
  - Given the input <6, 3, 1, 7>, the algorithm should produce <1, 3, 6, 7>
    - Called an instance of the problem

Sorting

- ***Sorting*** is a process that organizes a collection of data into either ascending or descending order.
- We encounter sorting almost everywhere:
  - Sorting prices from lowest to highest
  - Sorting flights from earliest to latest
  - Sorting grades from highest to lowest
  - Sorting songs based on artist, album, playlist, etc.

Sorting Algorithms

- There are many sorting algorithms (as of 27.10.2014 there are 44 Wikipedia entries)
- In this class we will learn:
  - Selection Sort
  - Insertion Sort
  - Bubble Sort
  - Merge Sort
  - Quick Sort
    - Bogo sort and Sleep sort are some bad/slow algorithms
- These are among the most fundamental sorting algorithms

Sorting Algorithms

- As we learnt in the Analysis lecture (time complexity), a stupid approach uses up computing power faster than you might think.
- Sorting a million numbers:

$O(n^2)$ algorithm	$2n^2$ instructions	10^9 inst/second	2000 secs.
$O(n \lg n)$ algorithm	$100\ n \lg n$ instructions	10^7 inst/second	60 secs.

- Interactive graphics: Algorithms must terminate in 1/30 of a sec.

Sorting Algorithms

- As we learnt in the Analysis lecture (time complexity), a stupid approach uses up computing power faster than you might think.
- Sorting a million numbers:

$O(n^2)$ algorithm	$2n^2$ instructions	10^9 inst/second	2000 secs.
$O(n \lg n)$ algorithm	$100\ n \lg n$ instructions	10^7 inst/second	60 secs.

- https://youtu.be/kPRA0W1kECg

Selection Sort

- Partition the input list into a sorted and unsorted part (initially sorted part is empty)
- **Select** the smallest element and put it to the end of the sorted part
- Increase the size of the sorted part by one
- Repeat this n-1 times to sort a list of n elements

Visualization of Selection Sort

Sorted			Unsorted			
23	78	45	8	32	56	Original List
8	78	45	23	32	56	After pass 1
8	23	45	78	32	56	After pass 2
8	23	32	78	45	56	After pass 3
8	23	32	45	78	56	After pass 4
8	23	32	45	56	78	After pass 5

Selection Sort (cont.)

```
template <class Item>
void selectionSort(Item a[], int n) {
  for (int i = 0; i < n-1; i++) {
    int min = i;
    for (int j = i+1; j < n; j++)
       if (a[j] < a[min]) min = j;
    swap(a[i], a[min]);
  }
}

template < class Object>
void swap( Object &lhs, Object &rhs )
{
  Object tmp = lhs;
  lhs = rhs;
  rhs = tmp;
}
```

Selection Sort -- Analysis

- What is the complexity of selection sort?
- Does it have different best, average, and worst case complexities?

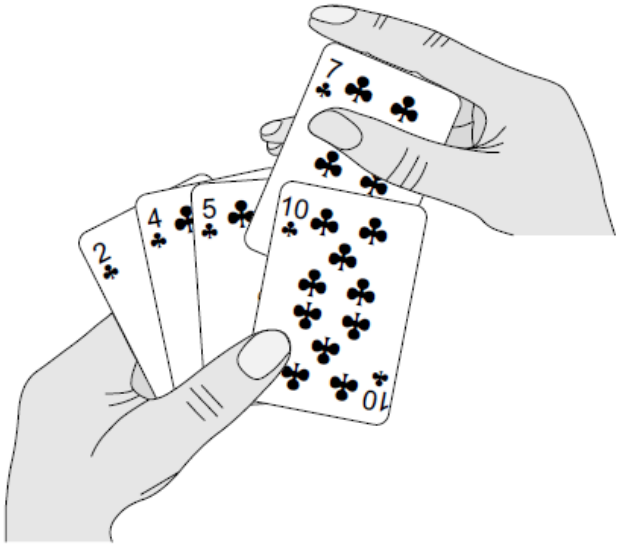
Selection Sort – Analysis (cont.)

- Selection sort is $O(n^2)$ for all three cases (prove this)
- Therefore it is not very efficient

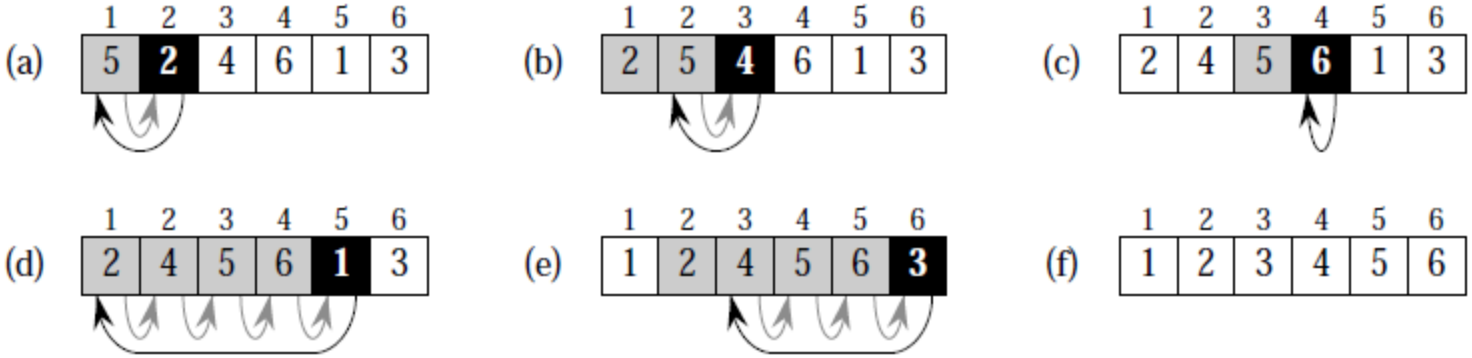
Insertion Sort

- Insertion sort is a simple sorting algorithm that is appropriate for small inputs.
  - Most common sorting technique used by card players.
- Again, the list is divided into two parts: sorted and unsorted.
- In each pass, the first element of the unsorted part is picked up, transferred to the sorted sublist, and **inserted** at the appropriate place.
- A list of n elements will take at most $n-1$ passes to sort the data.

Insertion Sort



Insertion Sort Visualization



Sorted			Unsorted			
23	78	45	8	32	56	Original List
23	78	45	8	32	56	After pass 1
23	45	78	8	32	56	After pass 2
8	23	45	78	32	56	After pass 3
8	23	32	45	78	56	After pass 4
8	23	32	45	56	78	After pass 5

Insertion Sort Algorithm

```
template <class Item>
void insertionSort(Item a[], int n)
{
   for (int i = 1; i < n; i++)
   {
      Item tmp = a[i]; // the element to be inserted
      int j;
      for (j=i; j>0 && tmp < a[j-1]; j--)
         a[j] = a[j-1]; // shift elements
      a[j] = tmp; // insert
   }
}
```

Insertion Sort – Analysis

- Running time depends on not only the size of the array but also the contents of the array.
- ***Best-case:*** ➔ **O(n)**
  - Array is already sorted in ascending order.
- ***Worst-case:*** ➔ **$O(n^2)$**
  - Array is in reverse order:
- ***Average-case:*** ➔ **$O(n^2)$**
  - We have to look at all possible initial data organizations.

Analysis of insertion sort

- Which running time will be used to characterize this algorithm?
  - Best, worst or average?
- Worst:
  - Longest running time (this is the upper limit for the algorithm)
  - It is guaranteed that the algorithm will not be worse than this.
- Sometimes we are interested in average case. But there are some problems with the average case.
  - It is difficult to figure out the average case. i.e. what is average input?
  - Are we going to assume all possible inputs are equally likely?
  - In fact for most algorithms average case is same as the worst case.

Bubble Sort

- Repeatedly swap adjacent elements that are out of order.
- https://youtu.be/vxENKlcs2Tw

- Sort pairs of elements far apart from each other, then progressively reduce the gap between elements to be compared. Starting with far apart elements, it can move some out-of-place elements into position faster than a simple nearest neighbor exchange. This generalization is called Shell Sort.

Bubble Sort

23	78	45	8	32	56
23	45	78	8	32	56
23	45	8	78	32	56
23	45	8	32	78	56
23	45	8	32	56	78
23	8	45	32	56	78
8	23	45	32	56	78
8	23	32	45	56	78

Bubble Sort Algorithm

```
template <class Item>
void bubleSort(Item a[], int n)
{
   bool sorted = false;
   int last = n-1;

   for (int i = 0; (i < last) && !sorted; i++){
      sorted = true;
      for (int j=last; j > i; j--)
         if (a[j-1] > a[j]{
            swap(a[j],a[j-1]);
            sorted = false; // signal exchange
         }
    }
}
```

```
BUBBLESORT(A)
for i ← 1 to length[A]
    do for j ← length[A] downto i + 1
        do if A[j] < A[j − 1]
            then exchange A[j] ↔ A[j − 1]
```

Bubble Sort – Analysis

- ***Best-case:*** ➔ **O(n)**
  - Array is already sorted in ascending order.
- ***Worst-case:*** ➔ **$O(n^2)$**
  - Array is in reverse order:
- ***Average-case:*** ➔ **$O(n^2)$**
  - We have to look at all possible initial data organizations.

- In fact, any sorting algorithm which sorts elements by swapping **adjacent** elements can be proved to have an $O(n^2)$ average case complexity

Theorem

- Any sorting algorithm which sorts elements by swapping **adjacent** elements can be proved to have an $O(n^2)$ average case complexity
- To understand this, consider the following array:

  [3, 2, 1]
- Here, we have three *inversions*: (3, 2), (3, 1), (2, 1)
- An inversion is defined as a pair (a, b) where a > b and index(a) < index(b)
- Note that a single swap of **adjacent** elements can only remove one inversion. Thus, for the above example we need 3 swaps to make the array sorted

Theorem

- Generalizing this, for an array of n elements, there can be C(n,2) total pairs (where C represents combination)
- C(n,2) = n(n-1) / 2
- We can assume that, on average, half of these pairs are inversions.
- Average number of inversions = n(n-1) / 4
- As each swap of adjacent elements can remove a single inversion, we need n(n-1) / 4 swaps to remove all inversions (that is to make the array sorted)
- The complexity of n(n-1) / 4 swaps is equal to $O(n^2)$
- This is the lower bound on the average case complexity of sorting algorithms that sort by swapping adjacent elements

Recursive Insertion Sort

- To sort A[1..n], we recursively sort A[1..n-1] and then insert A[n] into the sorted array A[1..n-1]
- A good practice of recursion
  - Code?

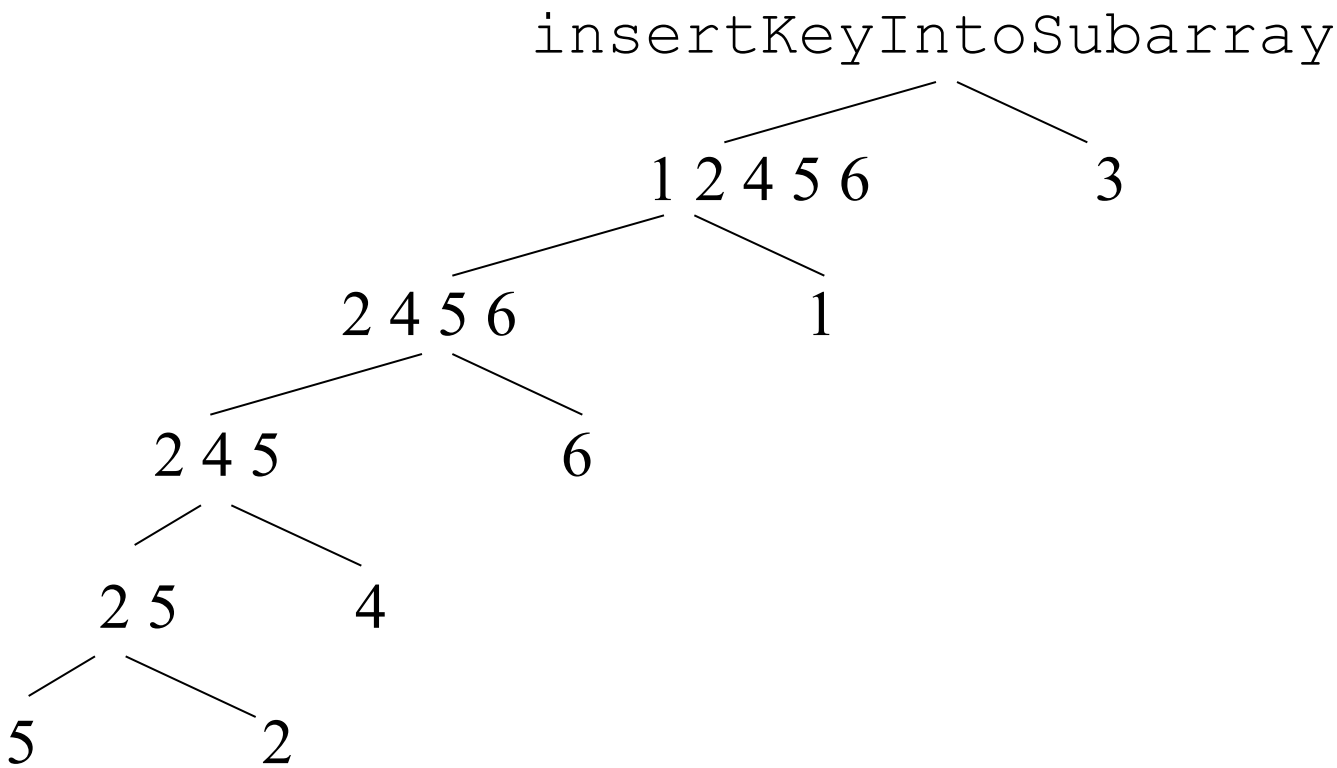
Recursive Insertion Sort

- To sort A[1..n], we recursively sort A[1..n-1] and then insert A[n] into the sorted array A[1..n-1]

```
insertionSortRec(A, n)
{
  if (n>1)
    insertionSortRec(A, n-1)
  insertKeyIntoSubarray(A[n], A[1..n-1]) //trivial; can
                                          //be done
  in                                        //O(n)
}
```

Recursive Insertion Sort

- To sort A[1..n], we recursively sort A[1..n-1] and then insert A[n] into the sorted array A[1..n-1]
- A = 5 2 4 6 1 3



Recursive Insertion Sort

- To sort A[1..n], we recursively sort A[1..n-1] and then insert A[n] into the sorted array A[1..n-1]
- $T(n) = T(n-1) + O(n)$ is the time complexity of this sorting

Recursive Insertion Sort

- To sort A[1..n], we recursively sort A[1..n-1] and then insert A[n] into the sorted array A[1..n-1]
- $T(n) = T(n-1) + O(n)$
  - Reduce the problem size to n-1 w/ O(n) extra work
  - Seems to be doing O(n) work n-1 times; so $O(n^2)$ guess?
  - $T(n) \leq cn^2$ //assume holds

$$
\begin{aligned}
T(n) &\leq c(n-1)^2 + dn \\
&= cn^2 - 2cn + c + dn \\
&= cn^2 - c(2n - 1) + dn \\
&\leq cn^2 \text{ //because large values of c dominates d}
\end{aligned}
$$

Recurrences

- How about the complexity of $T(n) = T(n/2) + O(1)$
  - Reduce the problem size to half w/ $O(1)$ extra work
  - Seems to be doing $O(1)$ work logn times; so $O(\log n)$ guess?
  - $T(n) \leq c\log n$ //assume holds

$$
\begin{aligned}
T(n) &\leq c(\log(n/2)) + d \\
&= c\log n - c\log 2 + d \\
&= c\log n - e \quad \text{//c can always selected to be > constant d} \\
&\leq c\log n
\end{aligned}
$$

Recurrences

- How about the complexity of $T(n) = 2T(n/2) + O(n)$
  - Reduce the problem to 2 half-sized problems w/ n extra work
  - Seems to be doing O(n) work logn times; so O(nlogn) guess?
  - $T(n) \leq cn\log n$ //assume holds

$$
\begin{aligned}
T(n) &\leq 2c(n/2 \log(n/2)) + dn \\
&= cn\log n - cn\log 2 + dn \\
&= cn\log n - n(c' + d) \quad //c' = c\log 2 \\
&\leq cn\log n
\end{aligned}
$$

Recurrences

- How about the complexity of $T(n) = T(n-1) + T(n-2)$
  - Reduce? the problem to a twice bigger one w/ no extra work
  - $T(n-1) + T(n-2) > 2T(n-2)$ ➔ n replaced by 2n-4 (doubled)
  - Or, n-size problem replaced with 2n-3 size problem (doubled)
  - Seems to be doubling the problem size; so $O(2^n)$ is good guess

Fast Algorithm

- Describe an algorithm that, given a set S of n integers and another integer x, determines whether or not there exists 2 elements in S whose sum is exactly x
  - $O(n^2)$ //brute force

Fast Algorithm

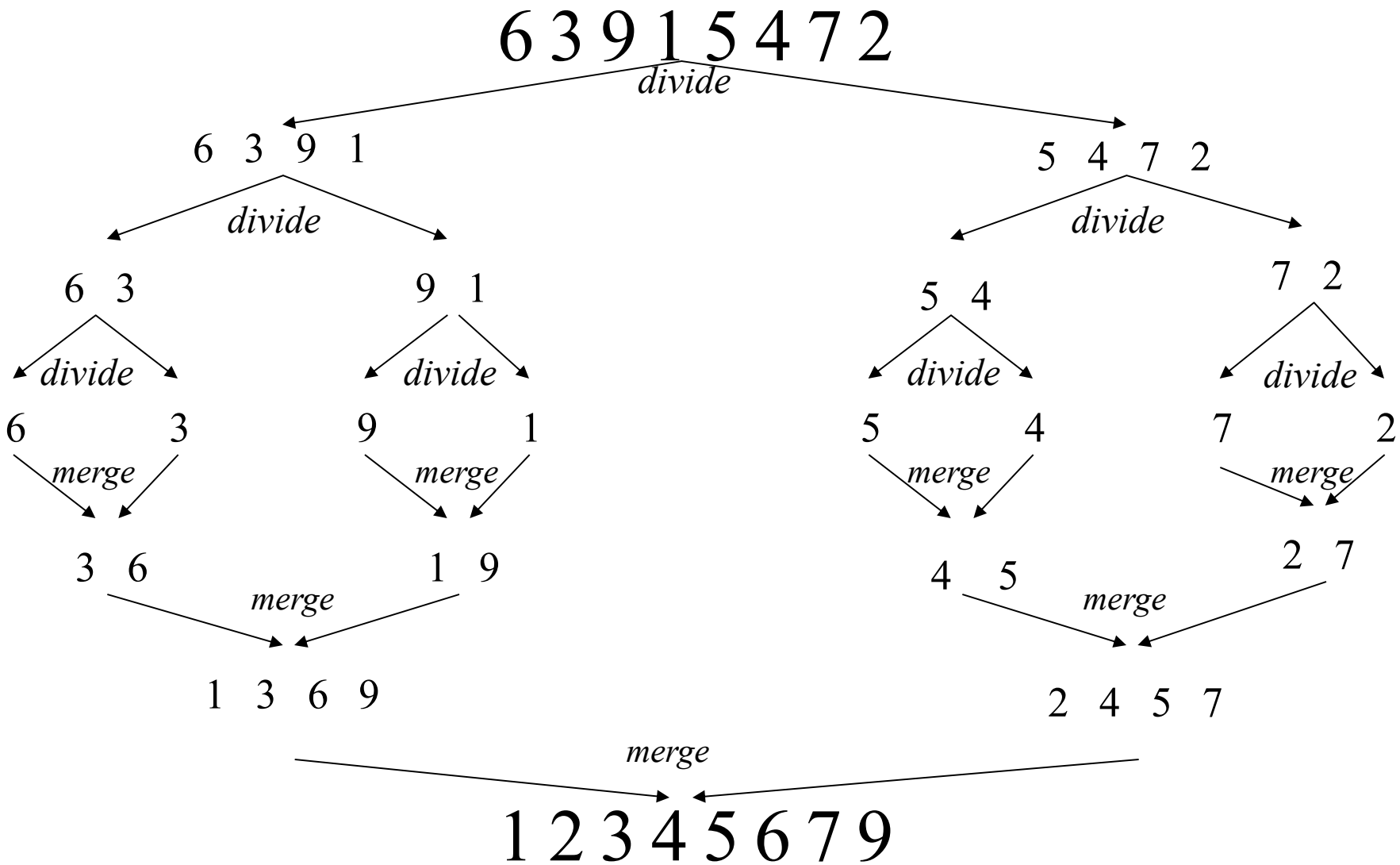
- Describe an algorithm that, given a set S of n integers and another integer x, determines whether or not there exists 2 elements in S whose sum is exactly x
  - O(nlogn) //sort followed by binary search

Mergesort

- Mergesort algorithm is one of two important divide-and-conquer sorting algorithms (the other one is quicksort).
- It is a recursive algorithm.
  - Divides the list into halves,
  - Sort each halve separately (recursively), and
  - Then merge the sorted halves into one sorted array.
- https://youtu.be/es2T6KY45cA

```
MERGE-SORT(A, p, r)
if p < r
    then q ← ⌊(p + r)/2⌋
         MERGE-SORT(A, p, q)
         MERGE-SORT(A, q + 1, r)
         MERGE(A, p, q, r)
```

Mergesort - Example



Merge

```
const int MAX_SIZE = maximum-number-of-items-in-array;
void merge(DataType theArray[], int first, int mid, int last)
{
   DataType tempArray[MAX_SIZE];    // temporary array
   int first1 = first;    // beginning of first subarray
   int last1 = mid;    // end of first subarray
   int first2 = mid + 1;  // beginning of second subarray
   int last2 = last;   // end of second subarray
   int index = first1; // next available location in tempArray

   for ( ; (first1 <= last1) && (first2 <= last2); ++index)
      if (theArray[first1] < theArray[first2])
         tempArray[index] = theArray[first1++];
      else
          tempArray[index] = theArray[first2++];
```

Merge (cont.)

```
   // finish off the first subarray, if necessary
   for (; first1 <= last1; ++first1, ++index)
      tempArray[index] = theArray[first1];

   // finish off the second subarray, if necessary
   for (; first2 <= last2; ++first2, ++index)
      tempArray[index] = theArray[first2];

   // copy the result back into the original array
   for (index = first; index <= last; ++index)
      theArray[index] = tempArray[index];
}
```

Mergesort

```
void mergesort(DataType theArray[], int first, int last) {
   if (first < last) { // more than one item
      int mid = (first + last)/2;   // index of midpoint
      mergesort(theArray, first, mid);
      mergesort(theArray, mid+1, last);

      // merge the two halves
      merge(theArray, first, mid, last);
   }
}  // end mergesort
```

Analysis of Mergesort

- What is the complexity of the merge operation for merging two lists of size n/2?
- It is O(n) as we need to copy all elements
- Then the complexity of mergesort can be defined using the following recurrence relation:

  $T(n) = 2T(n/2) + n$
- Solving this relation gives us O(nlogn) complexity //Slide 30
- The complexity is the same for the best, worst, and average cases
- The disadvantage of mergesort is that we need to use an extra array for the merge operation (not memory efficient)

Analysis of Mergesort

- Solving T(n) = 2T(n/2) + n gives us O(nlogn) complexity

Let's solve $T(n) = 2T(n/2) + n$ using substitution

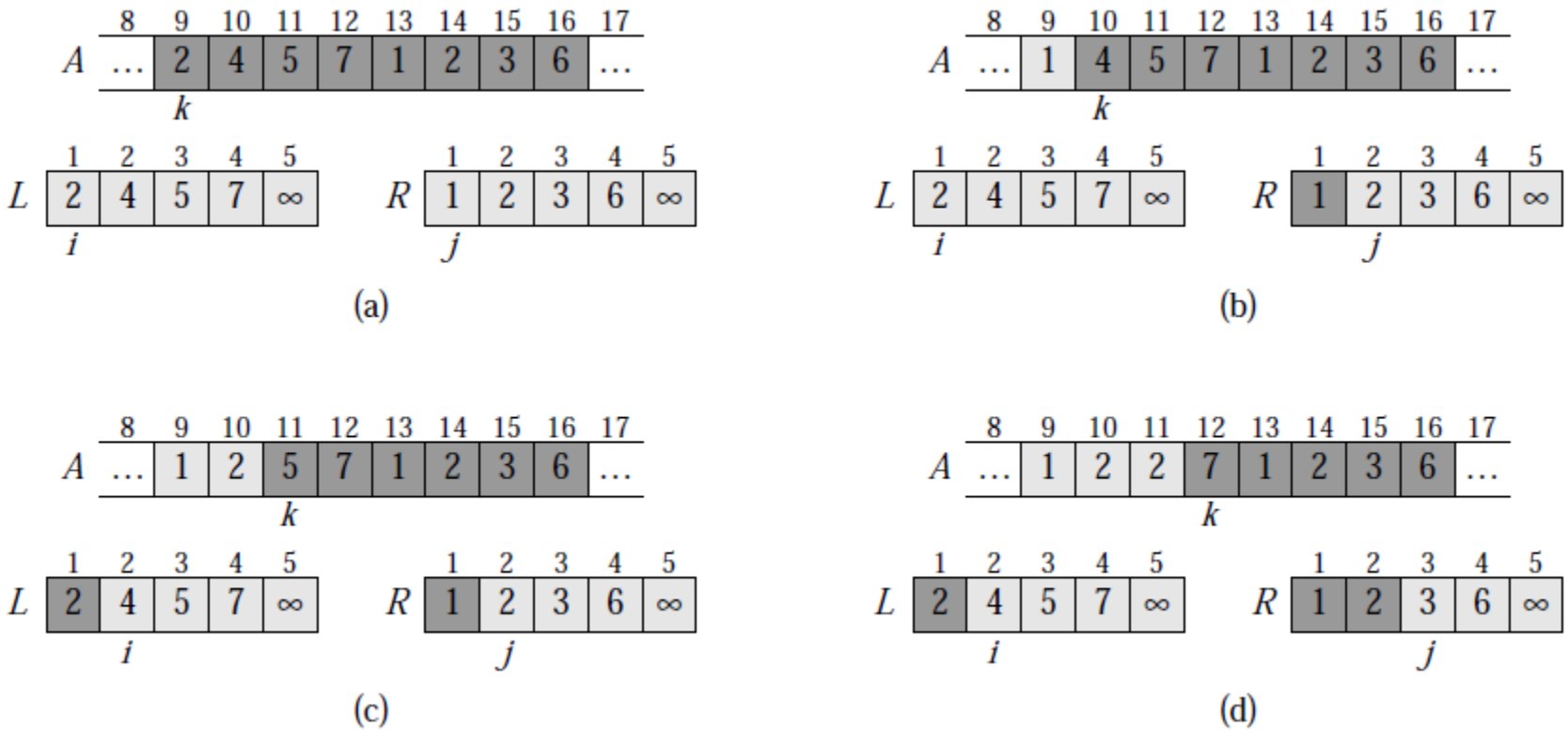
- Guess $T(n) \leq cn \log n$ for some constant c (that is, $T(n) = O(n \log n)$)
- Proof:
  - Base case: we need to show that our guess holds for some base case (not necessarily $n = 1$, some small n is ok). Ok, since function constant for small constant n.
  - Assume holds for $n/2$: $T(n/2) \leq c\frac{n}{2} \log \frac{n}{2}$ (Question: Why not $n - 1$?)
    Prove that holds for n: $T(n) \leq cn \log n$

$$
\begin{aligned}
T(n) &= 2T(n/2) + n \\
&\leq 2(c\frac{n}{2} \log \frac{n}{2}) + n \\
&= cn \log \frac{n}{2} + n \\
&= cn \log n - cn \log 2 + n \\
&= cn \log n - cn + n
\end{aligned}
$$

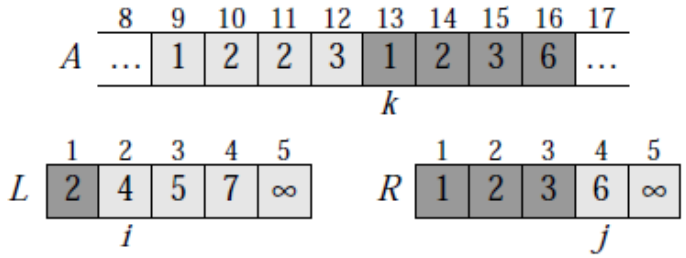
So ok if $c \geq 1$

Merging Step (Linear time)

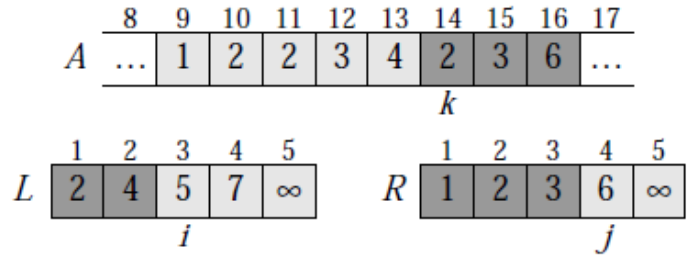
- Extra array for merge operation



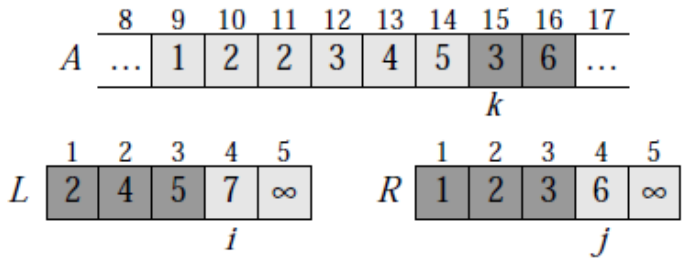
Merging Step (Linear time)



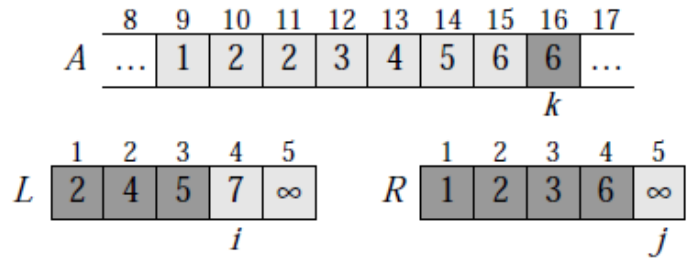
(e)



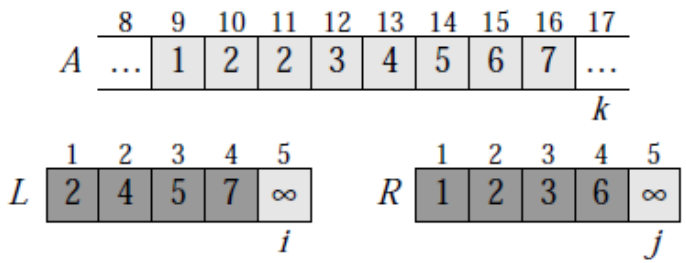
(f)



(g)



(h)

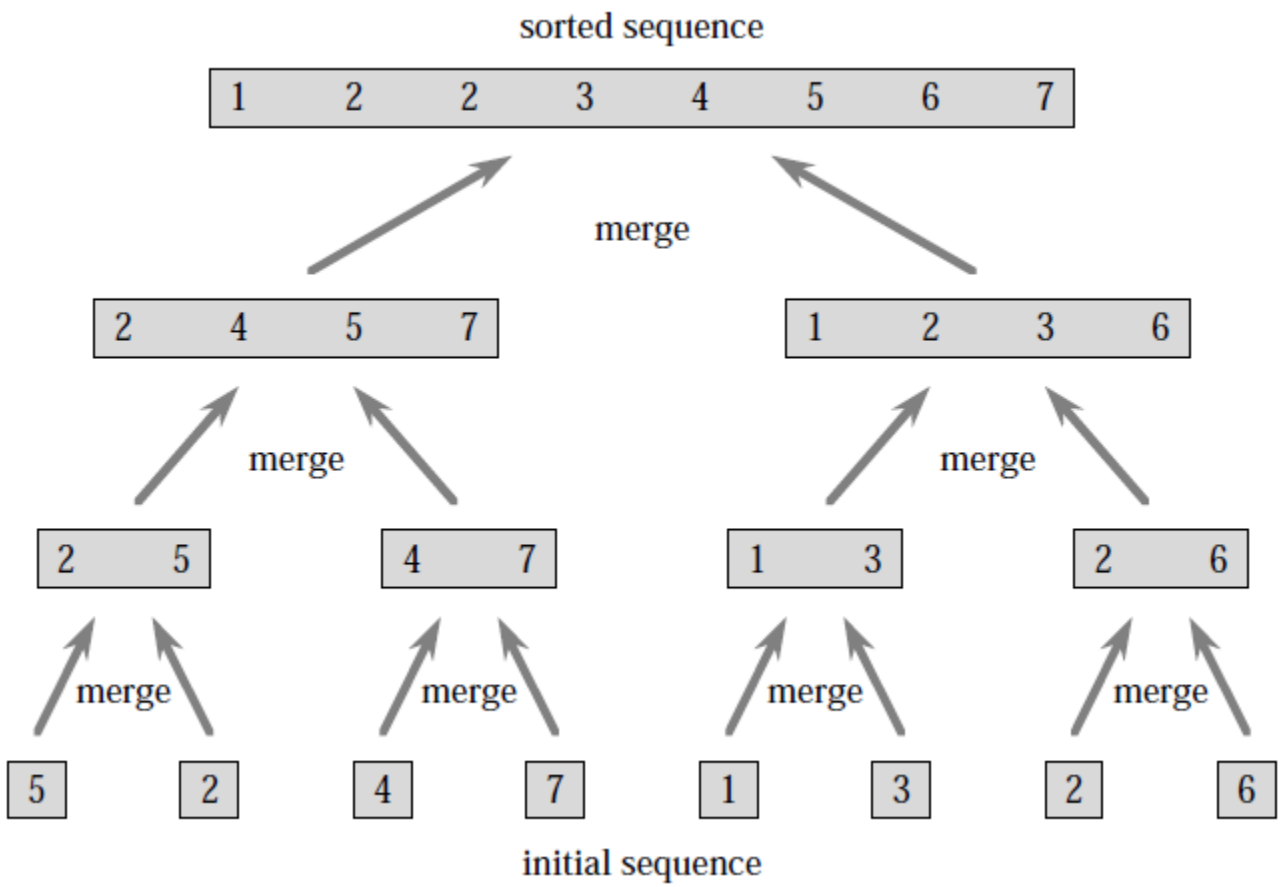


(i)

Demo

```
MERGE-SORT(A, p, r)
if p < r
    then q ← ⌊(p + r)/2⌋
         MERGE-SORT(A, p, q)
         MERGE-SORT(A, q + 1, r)
         MERGE(A, p, q, r)
```

- On array = {5, 2, 4, 7, 1, 3, 2, 6}

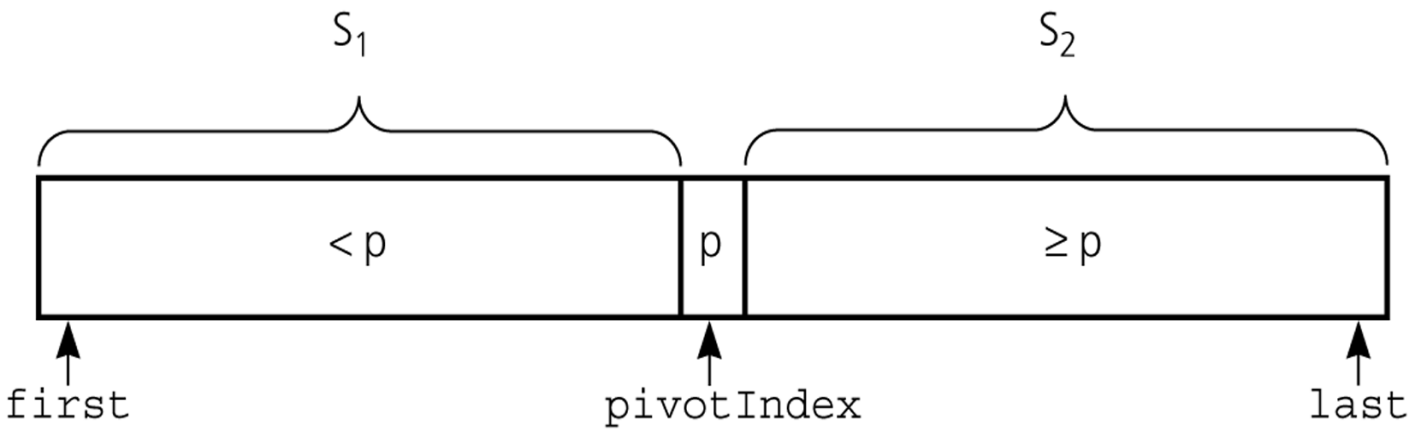


Quicksort

- Like mergesort, quicksort is also based on the *divide-and-conquer* paradigm.
- But it uses this technique in a somewhat opposite manner, as all the hard work is done *before* the recursive calls.
- It works as follows:
  1. First selects a pivot element,
  2. Then it partitions an array into two parts (elements smaller than and greater than or equal to the pivot)
  3. Then, it sorts the parts independently (recursively),
  4. Finally, it combines the sorted subsequences by a simple concatenation.
- https://youtu.be/vxENKlcs2Tw

Partition

- Partitioning places the pivot in its correct place position within the array.



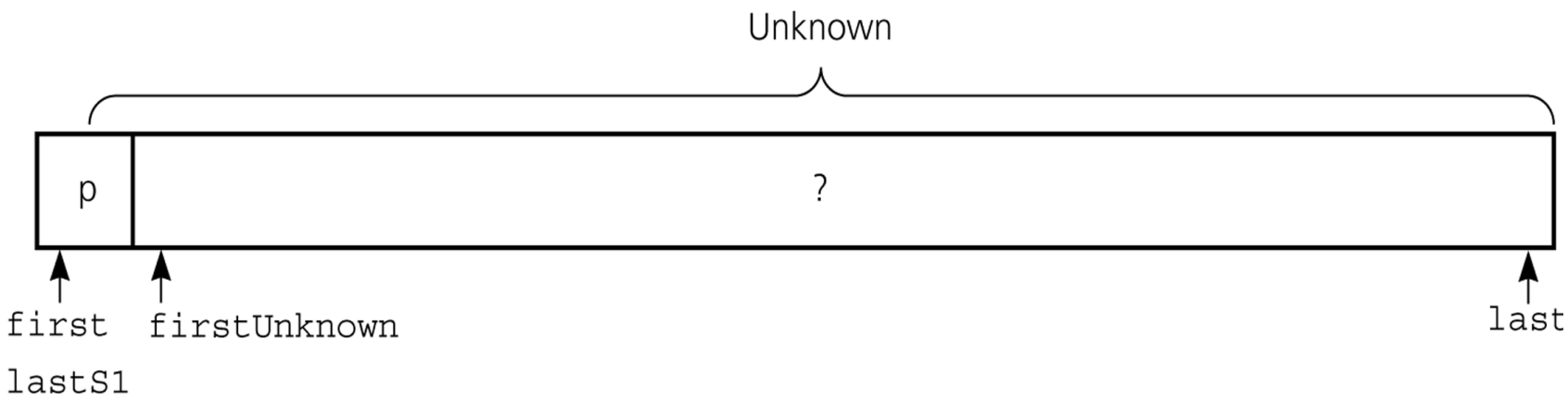
- Arranging the array elements around the pivot p generates two smaller sorting problems.
  - sort the left section of the array, and sort the right section of the array.
  - when these two smaller sorting problems are solved recursively, our bigger sorting problem is solved.

Pivot Selection

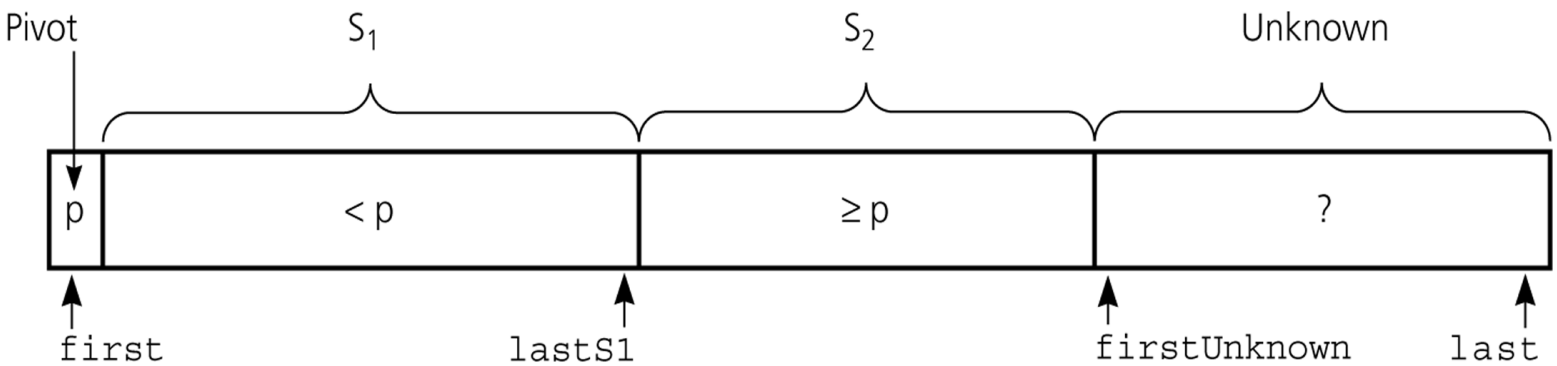
- Which array item should be selected as pivot?
  - Somehow we have to select a pivot, and we hope that we will get a good partitioning.
  - If the items in the array arranged randomly, we choose a pivot randomly.
  - We can choose the first or last element as the pivot (it may not give a good partitioning).
  - We can choose the middle element as the pivot
  - We can use a combination of the above to select the pivot (in each recursive call a different technique can be used)

Partitioning

Initial state of the array

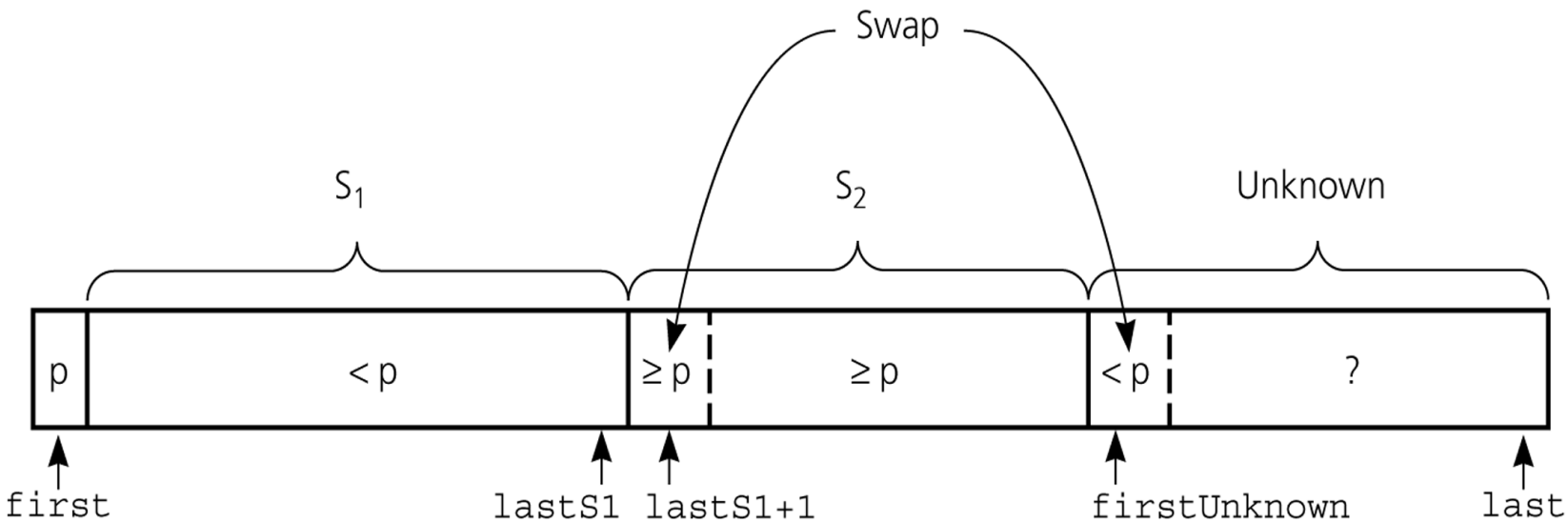


Partitioning



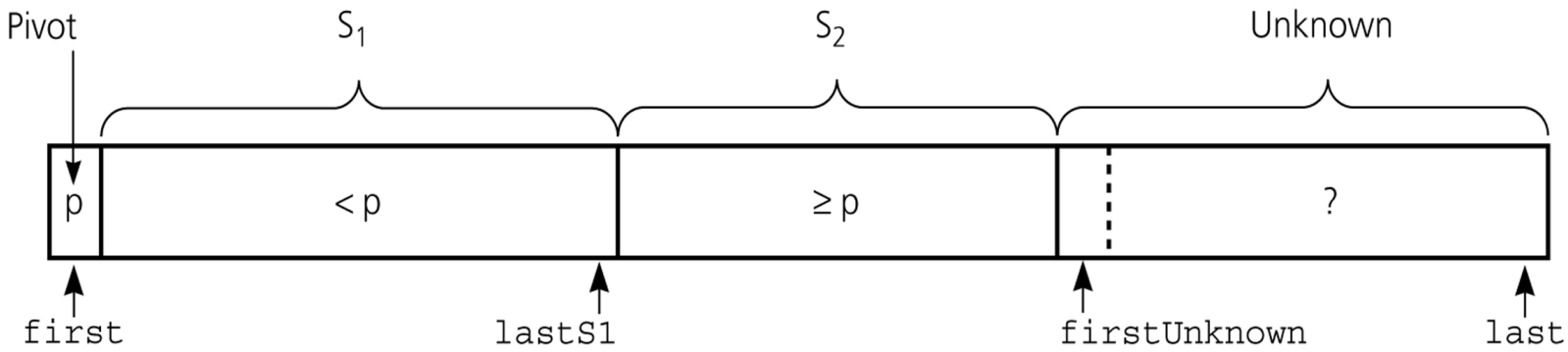
Partitioning

Moving `theArray[firstUnknown]` into S_1 by swapping it with `theArray[lastS1+1]` and by incrementing both `lastS1` and `firstUnknown`.

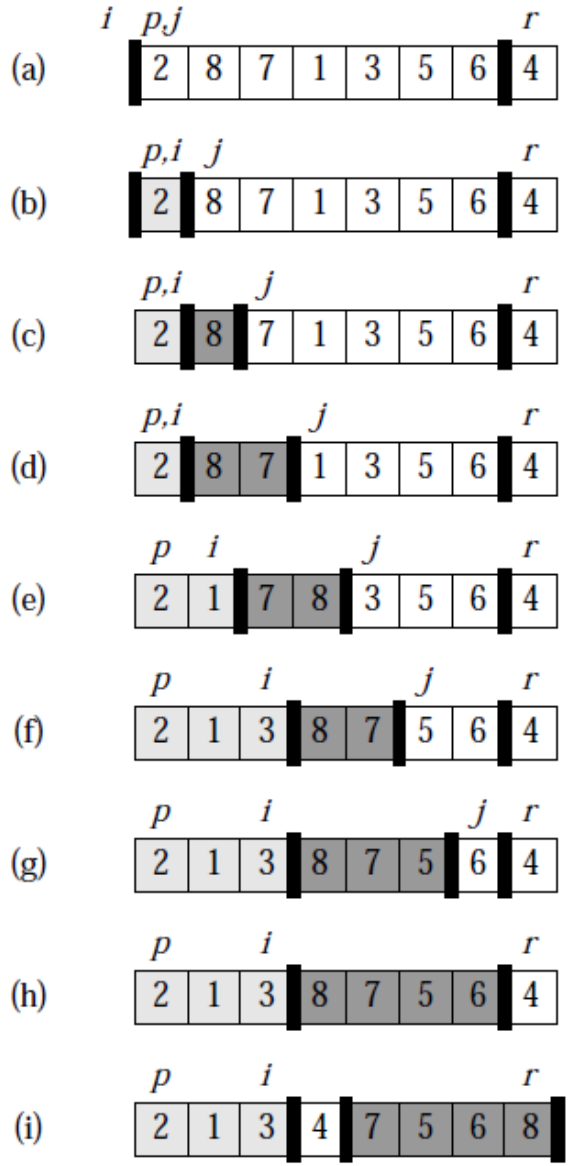


Partitioning

Moving `theArray[firstUnknown]` into S_2 by incrementing `firstUnknown`.



Partitioning – An Example



Partition Function

```
template <class DataType>
void partition(DataType theArray[], int first, int last,
        int &pivotIndex) {

   int pIndex = choosePivot(theArray, first, last);

   // put pivot at position first
   swap(theArray[pIndex], theArray[first]);

   DataType pivot = theArray[first]; // copy pivot
```

Partition Function (cont.)

```
   int lastS1 = first;            // index of last item in S1
   int firstUnknown = first + 1; //index of 1st item in unknown

   // move one item at a time until unknown region is empty
   for (; firstUnknown <= last; ++firstUnknown) {
      if (theArray[firstUnknown] < pivot)
         ++lastS1;
         swap(theArray[firstUnknown], theArray[lastS1]);
   }

   // place pivot in proper position and mark its location
   swap(theArray[first], theArray[lastS1]);
   pivotIndex = lastS1;
}
```

Quicksort Function

```
void quicksort(DataType theArray[], int first, int last) {

   int pivotIndex;
   if (first < last) {

      partition(theArray, first, last, pivotIndex);

      // sort regions S1 and S2
      quicksort(theArray, first, pivotIndex-1);
      quicksort(theArray, pivotIndex+1, last);
   }
}
```

Quicksort – Analysis

- If we always select the smallest or largest element as the pivot, we'll not be able to divide the array into similar sized partitions
- In that case, the complexity of the quicksort can be defined by:

  $T(n) = n + T(1) + T(n-1)$
- This gives $O(n^2)$ complexity (worst case) //Slide 28
- If our partitions are equal sized we have:

  $T(n) = n + 2T(n/2)$ (same as mergesort)
- This gives O(nlogn) complexity (best case)
- On average, quicksort has been proven to have O(nlogn) complexity as well (based on alternation b/w good/bad partitions)
- It also does not need an extra array like mergesort
- Therefore, it is the most popular sorting algorithm

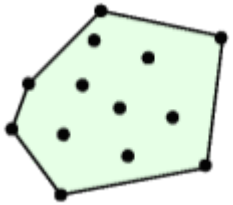
C/C++ Language Support

- We can implement sorting algorithms ourselves
- We can also use existing implementations
- In C, the function **qsort** (part of *stdlib.h* header) implements the quicksort algorithm
- In C++ **std::sort** (part of *algorithm* header) implements a mixture of quicksort, heapsort, and insertion sort

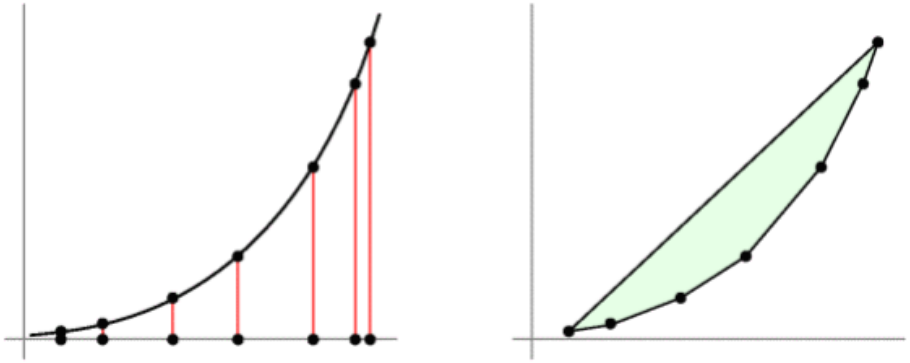
- Many sorting algorithms in comparison: https://youtu.be/ZZuD6iUe3Pc

Sorting vs. Geometry

- A cool reduction of Convex Hull problem into Sorting problem
- This is the convex hull of the point set P:
  - Release a large rubber band around points/nails



- Given unsorted a0, a1, a2, .., an, construct set of points in the plane via $ai=ai^2$ parabola as shown at left



- Every point must be on the convex hull ‘cos parabolas are convex
- Convex hull of this parabola = Ordered hull vertices = Sorting